Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Monterrey

Applying Optimal Control in MuJoCo

Implementación de Robótica Inteligente — TE3002B.101
RInClassAssignment

Carla Elizabeth Regalado Luna	A00837070
David Gilberto Lomelí Leal	A01571193
Abraham de Jesús Maldonado Mata	A00838581
Irvin David Ornelas García	A00839065
David Alejandro Soni Cuevas	A01571777

Catedrático: Nezih Nieto Gutiérrez

Fecha de entrega: 28 de abril de 2026

Abstract

We extend the PyMPC quadruped controller stack with a waypoint-based trajectory generator. The robot follows an ordered list of (x, y) goals using a rotate-then-advance state machine that emits a smooth velocity reference to the underlying Model Predictive Controller (MPC). To keep the robot upright over long sequences we layered a continuous speed governor, a slew-rate-limited command, and a predictive emergency brake on top of an adaptive speed cap. We validated the stack with the JAX-based sampling MPC on both CPU and GPU. The report documents the derivations, the controller integration, the calculations behind every threshold, and the evaluation protocol.

1. Problem Statement

Given:

- A quadrupedal robot with state $\mathbf{x} = [\mathbf{p}_b,\ \mathbf{R}_b,\ \mathbf{v}_b,\ \boldsymbol{\omega}_b,\ \mathbf{q},\ \dot{\mathbf{q}}]$, where $\mathbf{p}_b \in \mathbb{R}^3$ is the base position, $\mathbf{R}_b \in SO(3)$ its orientation, $\mathbf{v}_b, \boldsymbol{\omega}_b$ its linear/angular velocity, and $(\mathbf{q}, \dot{\mathbf{q}})$ the joint state.
- An ordered set of waypoints $\mathcal{W} = \{\mathbf{w}_0, \mathbf{w}_1, \ldots, \mathbf{w}_{N-1}\}$, $\mathbf{w}_i \in \mathbb{R}^2$.
- A tolerance pair $(\varepsilon_p, \varepsilon_\psi)$ for arrival distance and yaw alignment.

Goal: find a control law $\boldsymbol{\tau}(t)$ such that the base position visits each waypoint in order,

$$\|\mathbf{p}_b(t_i)_{xy} - \mathbf{w}_i\| < \varepsilon_p \quad \text{for some} \quad t_0 < t_1 < \cdots < t_{N-1},$$

and the robot remains upright ($|\theta(t)| < \theta_{\max}$, $|\phi(t)| < \phi_{\max}$) throughout, where θ, ϕ are pitch and roll.

This decomposes naturally into two subproblems:

1. **Trajectory generation.** Produce a base velocity reference $(\mathbf{v}_b^*, \boldsymbol{\omega}_b^*)(t)$ that drives $\mathbf{p}_b$ toward $\mathcal{W}$.
2. **Trajectory tracking.** PyMPC is responsible for choosing joint torques that follow $(\mathbf{v}_b^*, \boldsymbol{\omega}_b^*)$.

Our contribution lives entirely in (1).

2. Velocity Reference vs. Position Reference

The PyMPC controllers track base velocity, not base position. We chose to hand them a velocity reference rather than a position reference for three reasons:

(a) Mathematical compatibility. The single rigid body dynamics used by PyMPC's gradient MPCs and by the sampling MPC's centroidal model takes the form

$$\dot{\mathbf{p}}_b = \mathbf{v}_b, \qquad m\dot{\mathbf{v}}_b = \sum_i \mathbf{f}_i + m\mathbf{g}, \qquad I_b\dot{\boldsymbol{\omega}}_b = \sum_i (\mathbf{p}_i - \mathbf{p}_b) \times \mathbf{f}_i,$$

where $\mathbf{f}_i$ are the four ground reaction forces. Tracking $(\mathbf{v}_b^*, \boldsymbol{\omega}_b^*)$ is the controller's native objective.

(b) Bandwidth. A position reference would need an outer loop that converts position error to velocity, multiplying loop count and bandwidth budget. A direct velocity reference has the lowest possible latency.

(c) Saturation. The MPC has hard input bounds (friction cone, $f_z \geq 0$, torque limits). A position reference far from the current pose would saturate; a velocity reference is naturally bounded by `lin_vel`.

The drawback is that cross-track error during a long straight leg is not explicitly minimized. We mitigate this with a yaw-error feedback term during ADVANCING (see §4.4); a true outer position loop is left for future work.

3. Coordinate Frames

We use three frames:

- $\mathcal{F}_W$ — world (MuJoCo's global frame, $+z$ up).
- $\mathcal{F}_B$ — base (rigidly attached to the trunk).
- $\mathcal{F}_H$ — heading frame (rotation of $\mathcal{F}_W$ by the base yaw ψ about $+z$).

Inside `gym_quadruped`, `_ref_base_lin_vel_H` is stored in $\mathcal{F}_H$; `target_base_vel(frame='world')` returns $\mathbf{v}_b^W = R_z(\psi)\,\mathbf{v}_b^H$. Our WaypointNavigator outputs in $\mathcal{F}_W$ directly:

$$\mathbf{v}_b^{*,W} = c \begin{bmatrix} \cos\psi \\ \sin\psi \\ 0 \end{bmatrix}, \qquad \boldsymbol{\omega}_b^{*,W} = \begin{bmatrix} 0 \\ 0 \\ \omega_z \end{bmatrix},$$

where c is the scalar commanded forward speed and ω_z the commanded yaw rate. Note $\mathbf{v}_b^{*,W}$ always points along the heading: there is no sideways command. This matches the rotate-then-advance strategy — we never ask a trotting quadruped to translate sideways, which it does poorly.

4. Waypoint Navigator

The full per-step control flow follows a pipeline: check arrival → compute geometry ($\Delta = \mathrm{wp} - p_{xy}$, $d = \|\Delta\|$, $\psi_d = \mathrm{atan2}(\Delta_y, \Delta_x)$, $\psi_e = \mathrm{wrap}(\psi_d - \psi)$) → check waypoint proximity → run predictive brake → apply speed governor factors → slew-rate limit → output (v_W, ω_W).

4.1 Yaw Wrapping

Subtracting absolute yaws is unsafe near $\pm\pi$. We wrap the error:

$$\mathrm{wrap}(a) = \big((a+\pi) \bmod 2\pi\big) - \pi, \qquad \mathrm{wrap}(a) \in [-\pi, \pi).$$

This guarantees that a yaw error of 179° rotates by $-1^\circ \cdot k$, not $+359^\circ \cdot k$.

4.2 Slew-Rate Limiter

Given current command u_t, target u^*, accel cap a_+, decel cap a_-, and step Δt:

$$u_{t+1} = u_t + \mathrm{clip}\big(u^* - u_t,\ -a_-\Delta t,\ +a_+\Delta t\big).$$

We picked $a_+ = 0.3$ m/s^2 and $a_- = 2.0$ m/s^2. The asymmetry is deliberate:

- **Accel time** $0 \to 0.18$ m/s: $0.18/0.3 = 0.60$ s — trot has time to phase-stabilize.
- **Decel time** $0.18 \to 0$ m/s: $0.18/2.0 = 0.09$ s — faster than one trot half-cycle ($T/2 = 1/(2 \cdot 1.4) = 0.357$ s), so when the predictive brake fires we kill the command before the next foot strike.

4.3 Continuous Speed Governor

The desired forward speed is

$$v^* = v_{\max} \cdot \kappa \cdot f_{\text{dist}} \cdot f_{\text{yaw}} \cdot f_{\text{stab}},$$

with $v_{\max} = 0.18$ m/s (nominal) and $\kappa \in [0.25, 1.0]$ the adaptive cap (§4.5). Each factor lies in $[0, 1]$:

Factor	Formula	Threshold
f_{dist}	$\text{clip}(d/D_z,\, 0,\, 1)$	$D_z = 0.6$ m
f_{yaw}	$\text{clip}(1 - \lvert\psi_e\rvert/Y_b,\, 0,\, 1)$	$Y_b = 0.6$ rad
f_{stab}	$\min(r_\theta,\, r_\phi,\, r_{\dot\theta},\, r_{\dot\phi})$	see ramps below

The tilt ramps are saturating piecewise-linear:

$$r(x;\, e_-, e_+) = \begin{cases} 1, & |x| \le e_- \\ \dfrac{e_+ - |x|}{e_+ - e_-}, & e_- < |x| < e_+ \\ 0, & |x| \ge e_+ \end{cases}$$

with parameters:

Channel	e_- (EXIT)	e_+ (ENTER)	Slope width
Pitch θ	0.06 rad	0.22 rad	0.16 rad
Roll ϕ	0.08 rad	0.25 rad	0.17 rad
Pitch rate	0.6 rad/s	2.0 rad/s	1.4 rad/s
Roll rate	0.6 rad/s	2.0 rad/s	1.4 rad/s

Why these numbers? A trotting quadruped at 0.18 m/s pitches up to roughly $\pm 6^\circ \approx 0.10$ rad during normal stride load-shifting. We chose EXIT $= 0.06$ to not derate inside that nominal band and ENTER $= 0.22$ ($\approx 12.6^\circ$) as the start of "uncomfortable." The robot is empirically recoverable up to ≈ 0.30 rad, so we leave a 0.08 rad margin.

4.4 Yaw-Rate Command

$$\omega_z^* = \text{clip}\big(K_\psi\, \psi_e,\ -\omega_{\max},\ +\omega_{\max}\big) \cdot f_{\text{stab}},$$

with $K_\psi = 1.2\ \text{s}^{-1}$ and $\omega_{\max} = 0.35$ rad/s. A linear gain of 1.2 on ψ_e gives a closed-loop yaw time constant of $\tau_\psi \approx 1/K_\psi \approx 0.83$ s, which matches the trot stride time ($1/1.4 \approx 0.71$ s) — the yaw correction finishes within roughly one stride, avoiding overshoot.

The yaw-rate command also slews ($a_\pm = 3,\ 6$ rad/s^2).

4.5 Predictive Emergency Brake

The continuous governor is proportional to the current pitch — by the time $|\theta| \to \theta_{\max}$ the robot is already 0.1–0.3 s away from terminal fall (pitch rate $\dot\theta \approx 1$–3 rad/s during a fall). A purely current-state governor reacts too late.

We compute a one-horizon-ahead prediction:

$$\tilde\theta = \theta + \dot\theta \cdot H_{\text{pred}}, \qquad H_{\text{pred}} = 0.30 \text{ s}.$$

A trigger event fires when

$$|\tilde{\theta}| > \theta_{\text{hard}} \ \vee \ |\dot{\theta}| > \dot{\theta}_{\text{hard}} \ \vee \ |\theta| > \theta_{\text{hard}},$$

with $\theta_{\text{hard}} = 0.18$ rad $\approx 10.3°$ and $\dot{\theta}_{\text{hard}} = 1.6$ rad/s.

Why 0.30 s? Two reasons:

1. It must be longer than the slew-decel time (0.09 s, §4.2) so we have time to actually brake.
2. It must be shorter than the typical fall time at threshold θ_{hard}. A free pitching base falls under gravity-induced torque approximately as

$$\theta(t) \approx \theta_0 + \tfrac{1}{2}\alpha\, t^2, \qquad \alpha = \frac{mg\ell}{I_y} \approx 25 \text{ rad/s}^2$$

for Go2 (using $m = 15$ kg, $\ell \approx 0.10$ m COM offset, $I_y \approx 0.59$ kg m^2 from `config.py`). Reaching $\theta = 0.30$ from 0.18 takes $t \approx \sqrt{2 \cdot 0.12/25} \approx 0.10$ s. Predicting 0.30 s ahead is conservative — we trigger before the freefall regime begins.

When triggered:

- Force `brake_t` = 1.0 s of full stop.
- $\kappa \leftarrow \max(\kappa_{\min},\ \kappa \cdot 0.5)$ with $\kappa_{\min} = 0.25$.
- `cooldown_t` = 1.2 s to prevent re-trigger pile-up while the robot is settling.

Recovery: when stable ($|\theta| < \theta_{\text{exit}} \ \wedge \ |\dot{\theta}| < \dot{\theta}_{\text{exit}}$):

$$\kappa(t) = \min\big(1,\ \kappa + r_\kappa\, \Delta t\big), \qquad r_\kappa = 0.04 \text{ s}^{-1}.$$

Time to recover from $\kappa = 0.25$ to $\kappa = 1.0$: $\Delta\kappa / r_\kappa = 0.75/0.04 = 18.75$ s of uninterrupted stable operation. Each new trigger restarts the recovery from a halved cap, so the fixed point is the largest cap at which the gait is sustainable.

4.6 Adaptive Cap as a Contraction Map

Let κ_n be the cap immediately before the n-th trigger. With recovery period ΔT_n between consecutive triggers,

$$\kappa_{n+1} = 0.5 \cdot \min\big(1,\ \kappa_n + r_\kappa\, \Delta T_n\big).$$

Fixed points satisfy $\kappa^* = 0.5 \cdot \min(1,\ \kappa^* + r_\kappa \Delta T^*)$. If the gait is sustainable above some critical speed $\kappa_c\, v_{\max}$, triggers become rare ($\Delta T \to \infty$) and $\kappa \to 1$. If the gait is barely sustainable, triggers occur with period ΔT^* and the cap stabilizes at $\kappa^* = r_\kappa \Delta T^*$, i.e. an equilibrium where the robot trades cap erosion against time-based recovery.

This gives the navigator a self-tuning property: the longer the robot manages to walk without near-falls, the closer the operating speed approaches $v_{\max}$.

5. Spawn Fix and Reset Hardening

5.1 The 2.4 cm Offset

The default Go2 MJCF positions the trunk at $z = h_{\text{hip}} \approx 0.295$ m with the joint angles set so the foot tip sits at $z \approx 0.024$ m above the floor. At $t = 0^+$ the robot drops $\Delta z = 0.024$ m and strikes the ground at impact velocity

$$v_{\text{impact}} = \sqrt{2g\,\Delta z} = \sqrt{2 \cdot 9.81 \cdot 0.024} \approx 0.686 \text{ m/s}.$$

The impulsive spike easily pitches the base by several degrees, which is already enough to push the MPC's first-iteration sample distribution into a region where most rollouts terminate.

Fix:

```
env.mjModel.qpos0[2] -= 0.024
```

Now $v_{\text{impact}} = 0$ and the robot starts in static equilibrium.

5.2 Episode Reset Policy

`env.reset(random=True)` randomizes the spawn pose (position, yaw, joint configuration). When the robot has just terminated by hitting the floor, random spawn often lands it in another marginal pose and the next episode immediately terminates again. We reset with `random=False` so every retry starts from the same calibrated pose.

6. Controller Integration

6.1 Sampling MPC (Default, Validated)

The sampling controller solves, at each control instant, the centroidal optimization

$$\min_{\mathbf{u}_{0:H-1}} \sum_{k=0}^{H-1} \ell(\mathbf{x}_k, \mathbf{u}_k) + \ell_T(\mathbf{x}_H)$$

subject to centroidal SRB dynamics and friction-cone constraints, by generating M random control trajectories around a baseline, simulating each with JAX in parallel, and either picking the best (`random_sampling`) or doing a CEM/MPPI weighted average.

Selected parameters (`config.py`):

Parameter	**Value**
Horizon H	12 ($\times$ 0.02 s = 0.24 s)
`num_parallel_computations` M	10 000
Control parametrization	cubic_spline, 2 splines
Sampling method	random_sampling
σ	[0.2, 3, 10] (lin, ang, freq variance)

At $\Delta t_{\text{MPC}} = 0.01$ s (`mpc_frequency` = 100), the GPU amortizes the 10 000-rollout $\times$ 12-step JAX kernel comfortably. CPU is feasible but tighter; we observed real-time slips on CPU above $M = 15{,}000$.

6.2 Gradient MPCs (acados, Available)

`nominal`, `input_rates`, `lyapunov`, `kinodynamic`, `collaborative`. Each solves a quadratic SQP step every control instant via acados/HPIPM. They require building acados (one-off, see `installation/`).

The waypoint navigator works identically with any of these — the override happens at the velocity-reference level, before the MPC is invoked.

6.3 Trot Gait Kinematics

Step frequency $f_g = 1.4$ Hz, duty factor $D = 0.65$. Stride period $T_g = 1/f_g \approx 0.714$ s. Stance time $T_{\text{st}} = DT_g \approx 0.464$ s. Swing time $T_{\text{sw}} = (1 - D)T_g \approx 0.250$ s.

At commanded speed $v = 0.18$ m/s, the per-stride forward distance is

$$d_{\text{stride}} = v\,T_g = 0.18 \cdot 0.714 \approx 0.129 \text{ m}.$$

This is well below the kinematic limit ($\sim 0.5\,h_{\text{hip}} \approx 0.135$ m per swing leg), giving the optimizer slack.

7. Calculations Summary Table

Quantity	**Formula**	**Numerical value**
Spawn drop Δz	given	0.024 m
Spawn impact velocity	$\sqrt{2g\Delta z}$	0.686 m/s
Stride period	$1/f_g$	0.714 s
Stride length at $v_{\max}$	vT_g	0.129 m
Yaw-loop time constant	$1/K_\psi$	0.833 s
Slew accel time $0 \to v_{\max}$	$v_{\max}/a_+$	0.600 s
Slew decel time $v_{\max} \to 0$	$v_{\max}/a_-$	0.090 s
MPC horizon	$H\Delta t$	0.240 s
MPC step period	$1/f_{\text{MPC}}$	0.010 s
Predictive horizon H_{pred}	given	0.300 s
Falling time $0.18 \to 0.30$ rad	$\sqrt{2\Delta\theta/\alpha}$	≈ 0.098 s
Cap recovery $0.25 \to 1.0$	$0.75/r_\kappa$	18.75 s
Cap floor / nominal speed	$\kappa_{\min} v_{\max}$	0.045 m/s
Time per 2 m segment	L/v	≈ 11.1 s
Total nominal trajectory time	$4 \cdot 11.1 + 4 \cdot t_{\text{rotate}}$	≈ 50 s

For total trajectory time we used $t_{\text{rotate}} \approx \pi/(2\omega_{\max}) = \pi/(2 \cdot 0.35) \approx 4.5$ s for a 90° turn (worst case before slew). The 50 s estimate is the lower bound; in practice the speed governor and the adaptive cap stretch this by a factor depending on the gait stability margin observed.

8. Performance Evaluation

8.1 Metrics

Metric	Definition
Path completion	$\lvert\{i : \text{visited}\}\rvert/N$
Cross-track RMS	$\sqrt{\text{mean}_t\, d_\perp(\mathbf{p}_b(t),\, \overline{\mathbf{w}_{i-1}\mathbf{w}_i})^2}$ during ADVANCING
Final waypoint error	$\lVert\mathbf{p}_b(t_i) - \mathbf{w}_i\rVert$ at index advance
Pitch / roll statistics	mean, max, p_{95} over the run
Trigger events	count of predictive emergency-brake firings
Falls per minute	`is_terminated` rate
Wall-clock per MPC step	mean / max of `compute_actions` runtime
Effective forward speed	$\Delta x_{\text{world}}/t$ during ADVANCING

8.2 Recording Pipeline

`simulation.run_simulation(...)` accepts `recording_path`. When set, every step's state observables (`qpos`, `qvel`, base pose, base velocities, selected controller observables) are appended to an HDF5 dataset, ready for post-processing in pandas/numpy.

```
from simulation.simulation import run_simulation
from quadruped_pympc import config as cfg
run_simulation(qpympc_cfg=cfg, num_episodes=5,
               num_seconds_per_episode=120,
               recording_path='runs/sampling_trot/')
```

8.3 Comparison Protocol

For each controller in {sampling, nominal, input_rates, lyapunov}:

1. Run 5 episodes $\times$ 120 s with the same seed.
2. Record HDF5 trajectory.
3. Compute the metrics in §8.1.
4. Aggregate (mean $\pm$ std).

Suggested visualizations: top-down (x, y) trace with target waypoints overlaid; time series of $\theta, \phi, c, \omega_z$; histogram of cross-track error; bar chart of trigger events / falls.

8.4 Observed Qualitative Behavior

Configuration	Outcome
`lin_vel` = 0.25, no governor	Tips forward in 5–10 s
`lin_vel` = 0.25, governor only	Lasts 15–25 s, still tips
`lin_vel` = 0.18, governor + slew	Reaches first 1–3 waypoints, tips occasionally
`lin_vel` = 0.18, governor + slew + predictive brake + cap	Robust progression with periodic micro-stops; cap settles at the gait's sustainable speed

GPU vs. CPU: with `jax-cuda12-plugin`, the GPU kernel ran the 10 000-rollout × 12-step JAX program well within $\Delta t_{\mathrm{MPC}} = 10$ ms. CPU was feasible at the same M but with no headroom for raising H or M.

9. What Was Tried and Rejected

Attempt	Result	Why rejected
Crawl gait	MPC did not stabilize on either CPU or GPU at default sample budgets	Returned to trot
Zero-torque settle window	Legs went limp under gravity; worse than spawn drop	Removed
PD warmup with $K_p = 300$, $K_d = 8$ + gravity comp	Held the robot upright but transition to MPC was unstable	Removed
Binary STABILIZING supervisor (current state only)	Reacted too late; pitch already past hard limit when triggered	Replaced with predictive brake
World-frame velocity vs. base-frame velocity	Verified `target_base_vel(frame='world')` is world-frame; output now matches	Kept

10. Limitations and Future Work

1. **No outer position loop.** Cross-track accuracy depends on the yaw-error gain. A pure-pursuit / Stanley controller would reduce RMS cross-track error during long straight legs.

2. **One-way adaptive cap.** κ ratchets down on triggers; recovery is purely time-based. A reward-driven or bandit-style update would increase it on observed stability streaks rather than just letting time pass.

3. **No disturbance-rejection benchmarks.** PyMPC's environment exposes `ground_friction_coeff` and `external_disturbances_kwargs` — exercising these would test robustness more thoroughly.

4. **No comparison plots filled in.** The recording pipeline is wired and the metric set defined; populating the comparison across all four controllers requires building acados.

5. **Pitch prediction is first-order only.** A second-order predictor $\tilde{\theta} = \theta + \dot{\theta}H + \frac{1}{2}\ddot{\theta}H^2$ would reduce false triggers on transient motion but requires a clean estimate of $\ddot{\theta}$.

11. Reproducibility

```
# Python 3.12, project venv at .venv/
python3.12 -m venv .venv
source .venv/bin/activate
pip install -e .
pip install -U "jax[cuda12]"   # GPU support; CUDA bundled
python simulation/simulation.py
```

Tested on Ubuntu 24, kernel 6.8, NVIDIA GPU, jax 0.10 + jaxlib 0.10 + jax-cuda12-plugin 0.10. `jax.devices()` reports `[CudaDevice(id=0)]`.

Toggle GPU ↔ CPU in `quadruped_pympc/config.py`:

```
mpc_params['device'] = 'gpu'  # or 'cpu'
```

To regenerate this PDF after editing the report:

```
.venv/bin/python build_pdf.py
```